SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

Name : Atharva Ambilwade
PRN : 25070122055
Class : CSE
Batch : 2025-29

Assignment : 26

Subject: Programming with Java

ASSIGNMENT No : 26

TITLE : Write a Java program to create and execute multiple threads using the Runnable interface.

Problem Statement : Write a Java program to create and execute multiple threads using the Runnable interface.

Learning Objectives : To create multiple threads using the Runnable interface.

Theory :

The Runnable interface provides an alternative approach to creating threads in Java. A class implementing Runnable defines the run() method that contains the code executed by the thread. The Runnable object is passed to a Thread object, which starts execution using the start() method. Unlike extending the Thread class, implementing Runnable allows a class to inherit from another superclass, making the design more flexible. Runnable separates task definition from thread execution, resulting in better code organization and reusability. It is widely used in multithreaded enterprise applications and thread pool implementations.

Code :

A26.java > Language Support for Java(TM) by Red Hat > A26

```java
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " prints " + i);

            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}
public class A26 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        MyTask task = new MyTask();

        Thread t1 = new Thread(task, "Thread A");
        Thread t2 = new Thread(task, "Thread B");

        t1.start();
        t2.start();
    }
}
```

Output :

```
PS C:\Users\LENOVO\Documents\JAVA Assignments> cd
Thread B prints 1
Thread A prints 1
Thread A prints 2
Thread B prints 2
Thread A prints 3
Thread B prints 3
PS C:\Users\LENOVO\Documents\JAVA Assignments>
```

Conclusion : This program demonstrates multithreading using the Runnable interface. Two threads execute the same task concurrently, print numbers from 1 to 3, and pause briefly between iterations using Thread.sleep().

Exercise

1. **Create a program using Runnable to run two tasks simultaneously.**

Ans) Code :

A26E1.java > Java > A26E1

```java
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName()
                    + " is working: " + i);
        }
    }
}
public class A26E1 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyTask(), "Downloading");
        Thread t2 = new Thread(new MyTask(), "Uploading");

        t1.start();
        t2.start();
    }
}
```

Output :

```
PS C:\Users\LENOVO\Documents\JAVA Assignments> cd
Uploading is working: 1
Uploading is working: 2
Uploading is working: 3
Downloading is working: 1
Downloading is working: 2
Downloading is working: 3
PS C:\Users\LENOVO\Documents\JAVA Assignments>
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

Ans) Code :

A26E2.java > Language Support for Java(TM) by Red Hat > A26E2

```java
class Bank {
    int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
        System.out.println("Balance: " + balance);
    }
}

public class A26E2 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Bank b = new Bank();

        Runnable deposit = () -> b.deposit(500);
        Runnable withdraw = () -> b.withdraw(200);

        Thread t1 = new Thread(deposit, "Deposit Thread");
        Thread t2 = new Thread(withdraw, "Withdraw Thread");

        t1.start();
        t2.start();
    }
}
```

Output :

```
PS C:\Users\LENOVO\Documents\JAVA Assignments> cd
Withdrawn: 200
Balance: 800
Deposited: 500
Balance: 1300
PS C:\Users\LENOVO\Documents\JAVA Assignments>
```